

Universidad Tecnológica Nacional (UTN) Tecnicatura Universitaria en Programación a Distancia
Materia: Programación 1 **Trabajo Práctico Integrador (TPI)** **Título:** Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Fecha de Entrega: 15/06/2026

Integrantes:

- Alejandro Nicolás Abrigo
- Sofia Anahí Rosales

Contenido

Marco Teórico.....	1
Objetivos.....	2
Metodología	3
Desarrollo del Caso Práctico.....	3
Resultados y Conclusiones	5
Bibliografía y Enlaces	6

Marco Teórico

El presente proyecto se fundamenta en la aplicación práctica de diversas estructuras de datos y principios de programación en Python. A continuación, se detallan los conceptos clave utilizados y su aplicación:

Listas y Diccionarios: Las listas son estructuras de datos mutables y ordenadas que permiten almacenar colecciones de elementos. En Python, se definen con corchetes. Los diccionarios, por su parte, son colecciones no ordenadas que almacenan datos en pares de clave-valor, definidos con llaves. Ejemplo práctico en el proyecto: Utilizamos una lista de diccionarios para contener todo el dataset. Cada país es un diccionario individual. Esto nos permitió iterar sobre la lista principal y acceder a los datos de cada país de forma semántica usando sus claves, en lugar de recordar índices numéricos.

Modularización y Funciones: La modularización consiste en dividir un programa complejo en subprogramas más pequeños y manejables llamados funciones, definidos en Python con la palabra reservada `def`. Esto responde al principio de responsabilidad única. Ejemplo práctico en el proyecto: En lugar de escribir todo el código dentro de un gran bloque secuencial, creamos funciones específicas e independientes como `calcular_poblacion_promedio`. Esta función recibe los datos, hace el cálculo y retorna un

valor flotante. De esta manera, si hay un error en el cálculo estadístico, sabemos exactamente a qué función ir sin alterar el funcionamiento del menú principal o del filtrado.

Ordenamientos y Funciones Lambda: Python ofrece el método nativo `sort` para ordenar listas in situ. Para estructuras complejas, se utiliza el parámetro `key` combinado con una función anónima `lambda`, la cual permite definir la lógica de ordenamiento en una sola línea. Ejemplo práctico en el proyecto: Para ordenar los países por población de mayor a menor, utilizamos la instrucción de ordenamiento por la clave de población de manera reversa. La función `lambda` le indica al algoritmo que debe evaluar exclusivamente el valor numérico alojado en la clave `poblacion` de cada diccionario para decidir la posición final del elemento.

Manejo de Archivos CSV: La persistencia de datos se gestiona mediante archivos CSV. Python incluye la librería estándar `csv` para facilitar su lectura y escritura, transformando las filas de texto plano en estructuras de datos manejables. Ejemplo práctico en el proyecto: Utilizamos la clase `DictReader` para la carga inicial, la cual automáticamente convierte cada fila del archivo en un diccionario usando la primera fila como claves. Para el guardado de datos nuevos, implementamos `DictWriter`, iterando sobre nuestra lista de diccionarios y reescribiendo el archivo físico de países de manera estructurada para asegurar que los cambios persistan tras cerrar la consola.

Objetivos

Objetivo General:

Desarrollar una aplicación en consola mediante Python 3 que permita la gestión eficiente, persistencia y análisis estadístico de un dataset de países, aplicando los fundamentos de programación estructurada y modular.

Objetivos Específicos:

- Implementar persistencia de datos bidireccional mediante la lectura y escritura de archivos físicos (CSV).
- Diseñar una arquitectura de datos basada en listas y diccionarios para facilitar el acceso y manipulación de la información.
- Modularizar el código fuente asignando responsabilidades únicas a cada función, asegurando la legibilidad y el mantenimiento.
- Incorporar validaciones estrictas y manejo de excepciones (`try-except`) para evitar fallos del sistema ante entradas incorrectas del usuario o archivos corruptos.

Metodología

Para el desarrollo de este proyecto, el equipo adoptó un enfoque de trabajo iterativo y colaborativo. La división de tareas se gestionó mediante un repositorio de control de versiones (Git/GitHub), permitiendo el trabajo asíncrono a través de ramas independientes (branches) para la creación de nuevas funcionalidades (ABM y estadísticas) sin comprometer la rama principal (main). Se implementó un esquema de pruebas continuas (*testing* manual); cada módulo (lectura, validación, ordenamiento) fue testeado en la consola inmediatamente después de su desarrollo, evitando acumular errores lógicos para el final del ciclo de vida del software.

Desarrollo del Caso Práctico

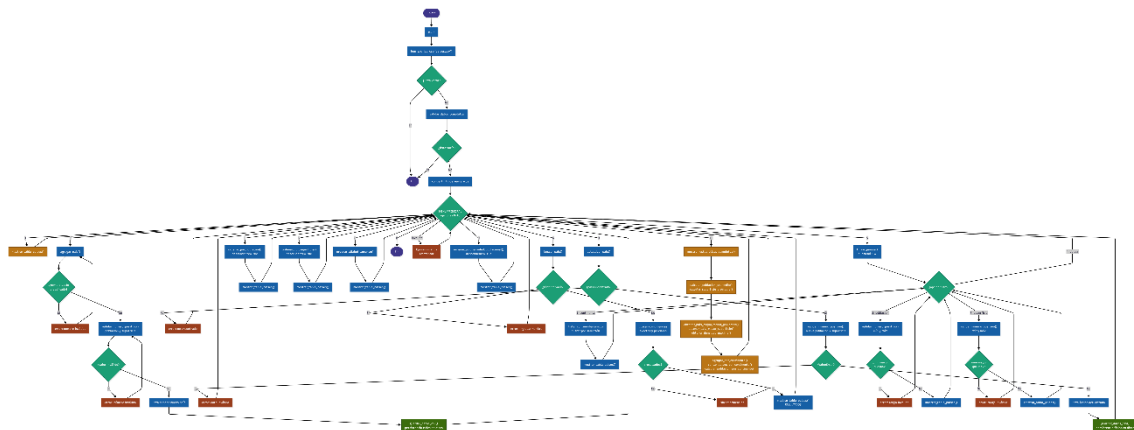
La principal decisión arquitectónica del proyecto fue la elección de una **lista de diccionarios** para gestionar el dataset en memoria. Se descartó el uso de listas paralelas o tuplas porque los diccionarios permiten modelar cada país como una entidad con atributos nombrados (clave-valor). Esto otorga dos grandes ventajas:

1. **Acceso semántico:** Es más legible invocar `pais["poblacion"]` que recordar el índice de una lista genérica (ej. `pais[1]`).
2. **Eficiencia en ordenamientos:** Al iterar sobre la lista principal, fue posible utilizar el método nativo `.sort()` combinado con funciones anónimas (lambda). Esto permitió ordenar dinámicamente todo el dataset utilizando cualquier clave del diccionario como criterio de evaluación en una sola línea de código, sin necesidad de implementar algoritmos de ordenamiento manuales menos eficientes.

Arquitectura de Módulos: Para cumplir con los estándares de buenas prácticas y correcta modularización, el sistema no solo fue dividido en funciones con responsabilidad única, sino que la arquitectura se distribuyó en múltiples archivos físicos (.py). Se definió un archivo `main.py` como orquestador central y punto de entrada del programa, delegando la carga lógica a cuatro módulos auxiliares independientes:

- **archivos.py:** Encargado exclusivamente de la lectura, validación de formatos y persistencia de datos en el archivo CSV.
- **analisis.py:** Contiene la lógica matemática, cálculos estadísticos y los algoritmos de ordenamiento mediante funciones lambda.
- **operaciones.py:** Gestiona la lógica de negocio (ABM, búsquedas y filtrados).
- **interfaz.py:** Aísla las funciones de presentación y despliegue visual en la consola.

Esta separación lógica elimina el anti-patrón de "código monolítico", previene importaciones circulares y facilita enormemente la lectura y el mantenimiento del sistema por parte del equipo.



[ciclo de vida sistema paises](#) (clickear para ver en detalle)

Seleccione una opción (0-10): 10

ESTADÍSTICAS GENERALES

DATOS GENERALES:

- Cantidad de países: 5
- Población total: 488,219,500 habitantes
- Superficie total: 12,787,266 km²
- Población promedio: 97,643,900 habitantes
- Superficie promedio: 2,557,453 km²

INFORMACIÓN POBLACIONAL:

- Mayor población: Brasil (213,993,437)
- Menor población: Chile (19,900,000)

INFORMACIÓN DE SUPERFICIE:

- Mayor superficie: Brasil (8,515,767 km²)

INFORMACIÓN DE DENSIDAD POBLACIONAL:

- Mayor densidad: Japon (332.83 hab/km²)

DISTRIBUCIÓN POR CONTINENTE:

- America: 3 país(es) - 279,270,200 habitantes
- Asia: 1 país(es) - 125,800,000 habitantes
- Europa: 1 país(es) - 83,149,300 habitantes

Seleccione una opción (0-10): 4

=====

BUSCAR PAÍS

=====

Ingrese el nombre (o parte del nombre): japon

✓ Se encontraron 1 resultado(s):

=====

RESULTADOS DE BÚSQUEDA

=====

País	Población	Superficie (km ²)	Continente	Densidad (hab/km ²)
Japon	125,800,000	377,975	Asia	332.83

=====

Seleccione una opción (0-10): 2

=====

AGREGAR NUEVO PAIS

=====

Nombre del país: Chile

Población: 19900000

Superficie (km²): 756102

Continente: America

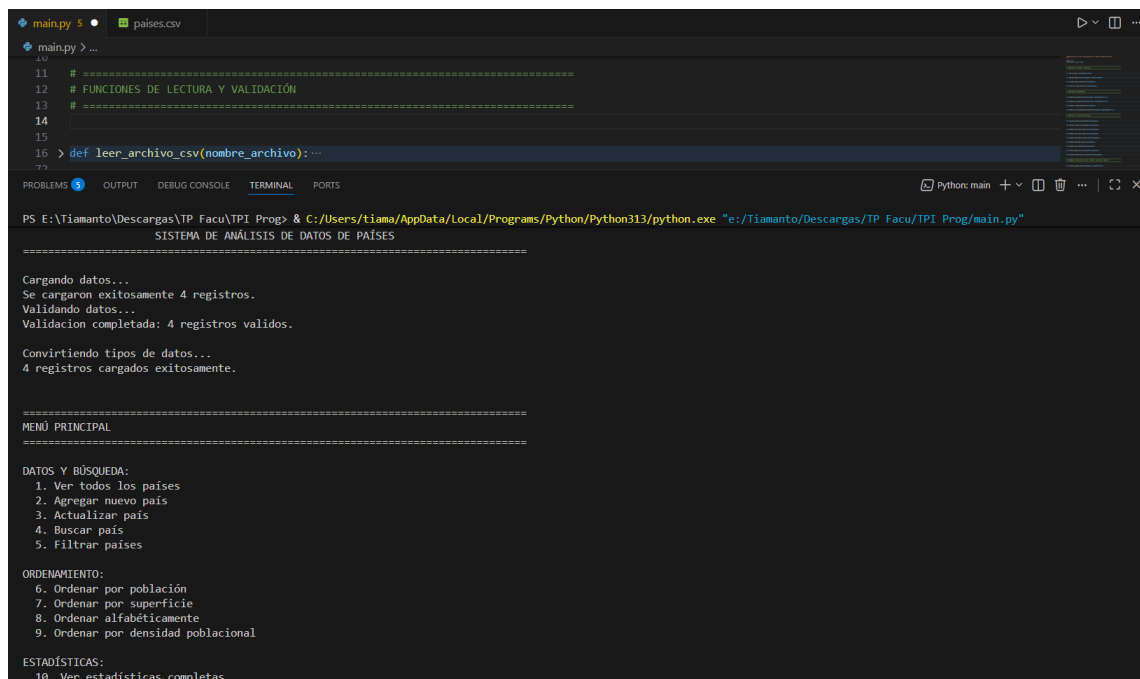
✓ País 'Chile' agregado exitosamente.

✓ Datos guardados exitosamente en 'países.csv'.

=====

MENÚ PRINCIPAL

=====



The screenshot shows a Python IDE with a file named 'main.py' open. The code in the file includes comments and a function definition for reading a CSV file. The terminal output shows the execution of the program, which displays a menu and processes data from a CSV file.

```
main.py 5 • países.csv
main.py > ...
11 # =====
12 # FUNCIONES DE LECTURA Y VALIDACIÓN
13 # =====
14
15
16 > def leer_archivo_csv(nombre_archivo):...
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

python: main + - - - - -

PS E:\Tiamanto\Descargas\TP Facu\TPI Prog> & C:/Users/tiama/AppData/Local/Programs/Python/Python313/python.exe "e:/Tiamanto/Descargas/TP Facu/TPI Prog/main.py"

SISTEMA DE ANÁLISIS DE DATOS DE PAÍSES

=====

Cargando datos...

Se cargaron exitosamente 4 registros.

Validando datos...

Validación completada: 4 registros validos.

Convirtiendo tipos de datos...

4 registros cargados exitosamente.

=====

MENÚ PRINCIPAL

=====

DATOS Y BÚSQUEDA:

1. Ver todos los países
2. Agregar nuevo país
3. Actualizar país
4. Buscar país
5. Filtrar países

ORDENAMIENTO:

6. Ordenar por población
7. Ordenar por superficie
8. Ordenar alfabéticamente
9. Ordenar por densidad poblacional

ESTADÍSTICAS:

10. Ver estadísticas completas

Resultados y Conclusiones

Reflexión técnica y desafíos: El desarrollo del sistema concluyó exitosamente, cumpliendo con la totalidad de los requerimientos funcionales. El mayor obstáculo técnico se presentó al implementar la persistencia de datos (guardado físico en el CSV). Inicialmente, los cambios solo se reflejaban en la memoria volátil. Esto se resolvió desarrollando un módulo específico utilizando `csv.DictWriter`, el cual sobrescribe el archivo original garantizando que las modificaciones del usuario no se pierdan al cerrar el programa. Como desarrolladores, este proyecto consolidó nuestra comprensión sobre cómo las estructuras de datos adecuadas simplifican drásticamente la lógica de filtrado y búsqueda.

Habilidades blandas y trabajo en equipo: La colaboración grupal fluyó de manera organizada. Mantuvimos una comunicación constante definiendo tiempos de entrega estrictos para cada módulo. Delegamos la lógica del ABM y validaciones a un integrante, mientras el otro se enfocó en las funciones estadísticas y de ordenamiento. La integración del código mediante *merges* cuidadosos en GitHub nos permitió experimentar de primera mano el flujo de trabajo estándar de la industria, evitando sobrescribir el trabajo del otro.

Bibliografía y Enlaces

Fuentes Consultadas:

- Python Software Foundation. (s.f.). *Data Structures*. Python 3.12 documentation. Recuperado de: <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. (s.f.). *csv — CSV File Reading and Writing*. Python 3.12 documentation. Recuperado de: <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (s.f.). *Sorting HOW TO*. Python 3.12 documentation. Recuperado de: <https://docs.python.org/3/howto/sorting.html>

Repositorio y Demostración:

- **Repositorio de GitHub:** <https://github.com/tiamant/gestion-datos-paises>
- **Video Demostrativo** [<https://youtu.be/OHl6K2mJC3w>]